

Implementation strategy: deploying within 90 days

Bellcourt Review Copilot. PACE stays; the copilot runs beside it.

When	What happens	Why / exit test
Day 0	Send Riverbend the 60-day written notice. Rescind UM-MEMO-2025-19. Re-review the 72 lumbar MRI denials, the 8 Kestrel bariatric denials and the MA glucose-monitor denials. Report timeliness from receipt, not keying.	The notice period is the critical path. The memo fix needs no technology.
Days 1 to 30: build and prove	Host in the Azure tenant that already has a signed agreement for patient data. Connect read-only: fax share, portal webhook, PACE read replica, eligibility files. Load all 38 plan documents into the registry with the account managers. Replay closed cases.	Exit: at least 95% agreement on a fresh, random QA sample of 200 closed cases, zero wrongful approvals.
Days 31 to 60: pilot on self-funded plans	8 senior nurses and 2 intake coordinators use it on employer queues. Intake sends the drafted fax-back within the hour. Measure minutes per case, agreement and override reasons weekly.	ERISA plans are not bound by the state AI laws or the notice clause. Exit: nurse minutes per case down by 25%.
Days 61 to 90: Riverbend go-live	Notice has run. Turn on for Riverbend with Arizona routing and the Texas disclosure in letters. Share audit-log extracts with Riverbend oversight. Train all reviewers.	Exit: MA on-time rate at least 97% and audit accuracy at least 95% for the month.
After day 90	Provider status page and requirements assistant, employer dashboard, policy-change alerts that re-check open cases.	Problems 2 and 3, on the same rule engine.

Who does what

- **Chief Medical Officer** owns the registry: a policy is not live until its effective dates are in it.
- **Account managers** file every plan amendment into the registry and copy UM Operations.
- **IT (4 people)** provides the read replica, fax share access and Azure hosting. No change to PACE.
- **QA team** audits a monthly sample of tool-assisted cases and owns the go/no-go numbers.
- **Staffing:** fill the 7.4 vacant nurse FTE rather than add 20. Recovering half of the 14.2 minutes spent finding rules is worth 9.3 FTE. License more medical directors in Arizona.

Design note: technology choices

One page. The stack is deliberately small so it can be explained, audited and replaced.

Choice	What I used	Why	In production at Bellcourt
Pattern	Agentic RAG with a deterministic rule resolver	Plain RAG retrieves by similarity and would return the retired version or the memo. An autonomous agent is not allowed to decide. So: code selects, the model reads and reasons, a person decides.	Same
Metadata registry	JSON built from each PDF's header plus a small overlay	Effective dates, client and authority rank are the missing piece in the UM Library. Filtering on them is what prevents wrong-rule errors.	Owned by the Clinical Policy Committee
Vector store	322 section embeddings (text-embedding-3-small, 256 dimensions) in a JSON file, cosine similarity in plain Python, fused with keyword rank	The corpus is 38 documents. A hosted vector database adds a key, a network hop and a failure point with no retrieval gain at this size. The index interface is one function.	Azure AI Search or Pinecone with the same metadata filters at 1,100+ documents
Model	google/gemini-2.5-flash, via OpenRouter or the Gemini API directly	Fast (about 6 seconds a case), cheap (about \$0.004 a case), reads fax images. One environment variable swaps the provider or the model; I tested Claude models too.	A model hosted in the Azure tenant under the existing agreement
Agent loop	Three to five calls: plan retrieval, review, verifier fixes, second check on approvals	Bounded and fully logged. Every step is visible in the audit trail.	Same
Verifier	Plain code	Citations must be in force, quotes must exist in the record, outcome must match the checklist, arithmetic on visit limits is recomputed. The model cannot talk its way past code.	Same
Backend	Python standard library only, two serverless functions	Zero dependencies means nothing to break at deploy time and a tiny attack surface.	Azure Functions or a container
Screen and access	One static HTML page. Sign-in with salted password hashes and signed 8-hour sessions; roles enforced on the server	Case data and fax images are served only to signed-in users. A nurse cannot sign an adverse decision; an auditor cannot run a review.	Single sign-on with Entra ID, same roles
Storage	Supabase (Postgres) through its REST API, one table for cases, letters and the audit log. Falls back to a local file	Serverless functions keep no state, so cases, decisions and letters need a database. One generic table keeps the prototype simple.	Azure SQL or Postgres inside the tenant, with proper tables
Letters and email	Letter text built in code from the human decision and the cited provision. A small built-in PDF writer. Gmail SMTP for the demo, delivered only to a configured demo mailbox	A letter is never generated by the model alone and never sent without a human decision. No extra dependency.	Bellcourt's correspondence system and print vendor
Evidence	Scripts that write JSON and the PDFs from the same run	Numbers in the documents cannot drift from the numbers in the app.	Monthly QA job

What I would change with more time: a database-backed audit log instead of browser storage; a fresh held-out audit sample; a stronger model for the review step if the fresh sample shows drift; plan documents for the other 32 employers.